

# Islands of Reach: Federated Content Delivery Across ISP-Level Network Partitions

Anonymous Author(s)

## Abstract

Network shutdowns are not a switch but a ladder: the 2024 disruption in Bangladesh combined throttling, platform blocking, TLS interference and a five-day national blackout. Systems built for the open internet stop at the first rung. We present a federated content substrate that keeps operating as connectivity degrades from global reach, through a national partition, to isolated ISP islands rejoined by a multi-homed bridge. Every mutation is a self-authenticating signed envelope validated by an identical nineteen-step pipeline whatever transport carried it, so peer trust bounds volume and never validity. Admission cost is charged at the origin and a per-peer quota protects the receiver, because node-local anti-abuse secrets cannot gate federated content. We evaluate on a container testbed with independent datastores and separate address spaces, reporting multi-hop propagation latency, per-scope path selection, resident memory and encoded message sizes against a measured JSON-LD federation baseline. We also report a partition-availability defect that only a deployed multi-peer testbed could expose — one unreachable peer stalled delivery to every reachable one, costing 407 s per crossing until it was isolated and bounded — and we state exactly which rungs of the ladder remain unmeasured, and why.

## CCS Concepts

• **Networks** → **Network reliability**; • **Security and privacy** → *Distributed systems security*.

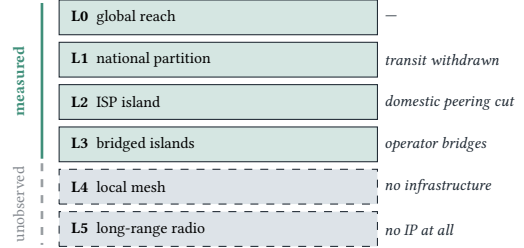
## Keywords

federation, network partitions, censorship resilience, delay-tolerant networking, transparency logs, canonical encoding

## 1 Introduction

Internet shutdowns are usually described as binary, and they are not. Measurements of the July–August 2024 disruption in Bangladesh record a sequence of distinct technical conditions [22]: selective throttling, blocking of individual platforms, interference with TLS handshakes, and finally a national blackout lasting five days, followed by a period in which domestic connectivity returned while international transit did not. Each of these breaks a different assumption, and a system that handles one may fail completely at the next.

We find it useful to name the conditions as rungs of a ladder. **L0** is unimpeded global reach. **L1** is a national partition: hosts inside the country can reach each other but not the outside. **L2** is an ISP island, where even domestic peering is severed and a network can reach only its own subscribers. **L3** is two or more such islands rejoined by an operator that holds transit on both. **L4** is a local segment with no infrastructure at all, and **L5** is long-range radio. Figure 1 summarises the ladder and marks how far our evaluation reaches.



**Figure 1: The resilience ladder. Each rung is cut by a different action, and a system that survives one may fail entirely at the next. This paper measures L0–L3. L4 and L5 are implemented but have never been observed end to end (§8), and no claim here rests on them.**

The gap this paper addresses is that the two mature families of systems sit at opposite ends of this ladder and neither spans it. Federated social systems — ActivityPub [26], Matrix [25], the AT Protocol [6] — assume L0 and degrade to unavailability at L1, because federation is defined in terms of reaching a named remote host. Delay-tolerant and mesh systems — DTN [7, 12], Serval [15], Bridgefy [2] — begin at L4 and provide no content, identity or moderation model of the kind a public forum needs. A population under a shutdown needs the middle of the ladder, which is where neither family operates.

We are explicit at the outset that the *mechanisms* we use are not new. Content addressed by the hash of its canonical encoding, signed at rest, is Nostr [13] and Secure Scuttlebutt [24]. Gossiped tree heads for detecting log rewriting is Certificate Transparency [18]. Separating credential issuance from credential verification is Privacy Pass [11]. Preferring the narrowest working path and keeping alternates warm is ICE [17] and MPTCP [14]. Our contribution is the composition, the invariants that composition forces, and what it costs when measured.

This paper contributes:

- **Canonicalisation as an *admission* predicate, and the three-position design space it completes.** Prior systems either canonicalise inside the verifier (Matrix) or refuse to define a canonical form at all (Cosmos SDK). We show these are two of three positions, take the third, and give the property it buys: no canonicalisation function is ever interposed between an attacker and a verifier, so a re-encoding relay cannot silently *repair* a malformed object into a verifiable one (§3.2).
- **Fork detection that stays safe when a consistency proof cannot be fetched.** The CT gossip literature resolves ambiguous tree-head pairs with a consistency proof. That proof is a round trip to the peer whose reachability is in doubt — unavailable exactly during a partition, which is when fork

detection is most likely to fire. We show the resulting false accusations are real and severe (one uplink flap falsely blocked two of three honest peers), give a strictly weaker rule that needs only a field the head already carries and no round trip, and show why the obvious implementation of it reintroduces the bug (§4.3).

- **Two federation invariants, each stated with the deployed failure that produced it:** an identifier inside a federated payload must be derivable by any node from the signed bytes, and admission cost must be charged at the origin because node-local anti-abuse secrets are meaningless elsewhere (§4).
- **A partition testbed, a resilience ladder that makes such systems comparable, and the first measurements of this one** — propagation swept over node count and queueing parameter, bridged crossing under a severed exchange, resident memory against a single-board-computer budget, encoded size against a measured ActivityPub baseline, and a denial-of-service surface measured rather than asserted (§6).

## 2 Related Work

Because this venue spans networking and security broadly, we introduce each concept rather than assuming it.

*Delay-tolerant networking.* DTN [12] and Bundle Protocol v7 [7] are the architectural ancestor of anything that must survive a partition: bundles are stored and forwarded over whatever bearer exists, with no assumption of an end-to-end path. BPv7 is the one prior system that covers our entire ladder, and we say so plainly in §7. What it does not supply is a content model, an identity model, or moderation. It is also weaker on integrity than is commonly assumed: BPv7 itself delegates all of it to BPsec [4], whose use on any individual bundle is optional even though implementing it is mandatory, and whose only default security context is HMAC-SHA2 with symmetric keys [5]. A default-context integrity block is therefore not verifiable by a third party.

*Mesh messaging.* Serval’s Rhizome [15] distributes signed bundles over ad-hoc links with no infrastructure whatsoever, and remains the strongest prior art for verification without a server: the bundle identifier is the verification key. Bridgefy carries Bluetooth mesh messaging and was broken comprehensively in 2020 [2]; its subsequent adoption of the Signal protocol fixed confidentiality but, by design, provides deniability rather than third-party verifiability [19], and broadcast messages remained unauthenticated in follow-up analysis [1].

*Federation over IP.* ActivityPub [26] is the dominant federation protocol. Notably, it makes neither object signatures nor HTTP Signatures normative — its authentication discussion is confined to a non-normative appendix which concedes that no strongly agreed mechanism existed at standardisation time. Interoperable authentication in deployed ActivityPub is therefore convention, and origin-validation defects follow from that [21]. Matrix [25] does sign events, over a Canonical JSON form. The AT Protocol [6] is the closest prior composition to ours: signed commits over a Merkle search tree, with deterministic CBOR encoding. It differs from us on two axes we care about, discussed in §7.

*Transparency logs.* Certificate Transparency [18] established the pattern we reuse: an append-only Merkle log [20] whose signed tree heads are gossiped, so that a participant which rewrites history is detectable by anyone comparing two heads. How to compare heads efficiently and safely is itself well studied — gossip protocols for verifying log consistency [8] and an aggregation-based design that discards a smaller head once one tree provably extends the other [10]. Both resolutions rest on obtaining a consistency proof; §4.3 is about what to do in the window where that proof cannot be fetched, which in a partitioned network is not a rare case.

*Path selection.* ICE [17] and MPTCP [14] establish that maintaining several candidate paths and continuously preferring among them is more robust than failing over on demand. We apply the same reasoning to reachability *scope* rather than to addresses.

## 3 Design

### 3.1 One write path

The system exposes exactly one write endpoint. A post, a vote, a moderation action, a key revocation and a broadcast are all the same structure: an *envelope* carrying a domain string, an author key, a timestamp, a priority class, an opaque body, and a signature. There is no per-feature write route, and adding a feature adds a registry row and a handler rather than a branch anywhere in the ingress path.

The envelope’s identifier is derived from its own bytes:

$$id = \text{BASE32}(\text{SHA-256}(\text{canonical}(\text{fields}_{1..12})))$$

and the Ed25519 [3] signature covers those same bytes. Nothing about this identifier depends on which node computed it, which is what allows two independent instances to agree on identity without coordinating.

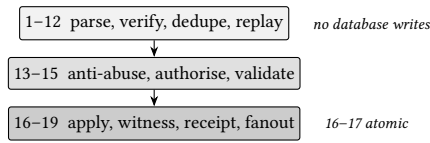
### 3.2 Canonical encoding as an admission gate

Canonical encoding is where our design differs most sharply from prior work, and the difference is not *whether* to canonicalise but *where in the pipeline* canonicalisation sits and what happens when it fails. Three positions exist in the literature.

Matrix canonicalises *at verification time*: the receiver strips signature fields, re-encodes the parsed object into Canonical JSON, and checks the signature against the result [25]. The verifier therefore checks a signature against bytes it produced itself, and an intermediary may re-serialise an event freely without invalidating it.

Cosmos SDK takes the opposite position and rejects canonicalisation outright, stating that verification “is based on comparing the raw TxBody and AuthInfo bytes ... not based on any ‘canonicalization’ algorithm which creates added complexity for clients in addition to preventing some forms of upgradeability” [9]. Whatever bytes the sender emitted are by construction the contract; there is nothing to reject.

We take the verification rule of the second and the insistence on a defined canonical form of the first, and order them differently. The signature is checked against **the bytes as they arrived**. Separately, and *before* verification, the decoder re-encodes the parsed message and compares; a mismatch is **rejected**, never repaired.



**Figure 2: The ingress pipeline. Steps 1–12 perform no writes, so an invalid-envelope flood cannot amplify into database load. Steps 16 and 17 are atomic with respect to each other: a projected envelope missing from the Merkle log would be a transparency failure.**

Canonicalisation is thus not in the verification path at all — it is an admission predicate:

Matrix canonicalises to decide *what to verify against*;  
Cosmos declines to canonicalise at all; we canonicalise  
to decide *what to admit*, and verify against what arrived.

The hazard this forecloses is concrete. Under canonicalise-at-verify, a re-encoding intermediary silently *repairs* a malformed object into a verifiable one; a prior version of this system was vulnerable to exactly that signature-confusion pattern. Under the Cosmos rule the hazard cannot arise because the question is never asked. Under ours it cannot arise because the answer is rejection.

Protobuf serialisation “is not (and cannot be) canonical” [16], and that guidance warns against byte-equality as a proxy for *message equality*, where differing bytes for one message give false negatives. We ask a narrower question — *is this the one accepted encoding?* — for which a second valid form is not an inconvenience but precisely the thing to reject. Protobuf declining to define a canonical form is why a profile must define one and prove it across implementations (§6.2).

We concede the cost that ADR-020 names: strictness impedes some forms of upgradeability. Unknown fields are not retained, so an old node cannot forward-carry a field it does not understand; the response is a version bump rather than an in-place contract edit.

### 3.3 The validation pipeline

Every envelope, from every transport, runs an identical nineteen-step pipeline (Figure 2). Two properties of its ordering carry most of the weight.

**Steps 1–12 perform no database writes.** Size, parse, version, domain, plane, algorithm policy, priority, clock, signature, certificate, dedupe and replay all complete before anything is persisted. An attacker flooding invalid envelopes therefore consumes CPU but cannot amplify into write load.

**Steps 16 and 17 are atomic with respect to each other.** Projection and the append to the Merkle log occur in one transaction. An envelope visible in a projection but absent from the log would be exactly the transparency failure the log exists to prevent, so the storage layer must support multi-document transactions; this is a real deployment constraint and not an implementation detail.

Each step is an independently testable pure function, and the pipeline is composed from them rather than written as one procedure.

## 4 Federating Across a Partition

Federation is server-to-server only. Clients do not speak it, because operators choose their peers and clients do not, and because the protocol’s connection profile degrades badly on lossy mobile links.

### 4.1 Four rules that carry the weight

*A peer’s bytes are never re-encoded.* The transport uses a passthrough codec. Because the decoder establishes canonicity by re-encoding and comparing (§3.2), any round trip through a generated codec in the network adapter would silently repair a non-canonical envelope from an untrusted peer, which would then validate — the precise hazard the admission gate exists to foreclose, arriving over the network and bypassing the gate.

*Trust bounds volume, never validity.* Inbound envelopes re-run all nineteen steps. A trusted peer’s forged envelope is rejected exactly as a stranger’s is. Trust determines quota and which priority classes a peer may push, and nothing else.

*Deduplication is a unique index, not a read-then-write.* The pair  $(content\_id, direction)$  is a uniqueness constraint in the database. A read-then-write check is unsound under the concurrency federation actually produces, where an overlapping activity stream and a backfill can deliver the same envelope simultaneously.

*An envelope is never relayed to its sender.* The pipeline carries the origin peer and excludes it from fanout. Content dedupe alone makes such a loop terminate, but it does not make it free: without this, every envelope pays a full round trip before being discarded.

### 4.2 Two invariants, and the failures that produced them

*An identifier in a federated payload must be derivable from the signed bytes.* An earlier version derived a community identifier from the projecting node’s own key. With one node this is indistinguishable from correct, because the projecting node is the origin node. With two, the same signed envelope produced different community identifiers on each instance and the projections diverged permanently. The rule that replaced it is a question asked before any identifier is minted: *would a different node, given only these bytes, compute the same value?*

*Admission cost is charged at the origin.* Proof-of-work, credit balances, blind credentials and epoch nullifiers are each keyed to one node deliberately. A proof minted at the origin is therefore not merely unverifiable elsewhere — it is meaningless elsewhere. Charging again on receipt makes every anti-abuse-gated domain unfederatable. Cost is paid once, at origin; the receiver’s protection is a per-peer, per-class quota with an explicit backpressure hint. Verification is unaffected: what is skipped is a payment, not a check.

A related consequence is that a peer exceeding quota is not disconnected. It would reconnect immediately, pay the handshake again, and arrive in the same state, so refusing costs more than accepting. The hint lets a well-behaved peer self-regulate, and a peer that ignores it is demoted.

### 4.3 Fork detection when a consistency proof is unavailable

A transparency log lets a participant detect a peer that rewrote history: tree heads are signed and gossiped, and a head smaller than one already seen suggests entries were removed. This is the Certificate Transparency construction [18] and we adopt it unchanged.

*What the gossip literature already settles.* Comparing tree heads pairwise is known to be subtle, and the CT gossip protocols address it directly: an STH-only gossip design observes that when one tree provably *extends* another, the smaller head can be discarded without reducing the chance of detecting an attack [10], and efficient gossip protocols for exactly this consistency check are well studied [8]. The resolution in that literature is a **consistency proof** between the two heads.

*What a partition changes.* A consistency proof must be fetched from the log. That is a round trip to the very peer whose reachability is in question — and in our setting the peer may be behind the cut that is being routed around, which is precisely when fork detection is most likely to fire. So the standard resolution is available exactly when it is least needed.

The failure this produces is not a missed detection but a false one, which is worse. In our node four independent code paths observe a peer’s head: the outbound handshake, the inbound handshake, a head carried on a peer record, and the gossip round. Each samples at a different instant and nothing sequences their arrival, so *gossip records size 8, then a reconnect delivers the size-6 head it captured moments earlier* is ordinary concurrency. Compared pairwise without a proof, it reads as a tree that shrank. Because a fork is the strongest signal the system has, the peer is demoted to a state only an operator can lift — so a node partitions itself from honest peers, during a partition, on evidence it manufactured. In our deployed four-node testbed one uplink flap produced this against **two of three peers at once**, both with the same manufactured reason (*tree shrank from 8 to 6*), and neither log had been rewritten.

*A rule that costs no round trip.* Signed tree heads already carry a signed timestamp, so the readings can be ordered without asking anyone anything. Let  $(t, n)$  be the timestamp and size of an incoming head and  $(t_0, n_0)$  the recorded one:

- (1) if  $t < t_0$  the reading is stale: return UNKNOWN, and **do not record it**;
- (2) otherwise if  $n < n_0$  the tree regressed: report FORKED;
- (3) otherwise record  $(t, n)$  and report OK.

This does not replace a consistency proof — it is strictly weaker, and a node that can reach the peer should still fetch one. What it does is remove the false accusations in the window where no proof can be obtained, which is the window that matters here.

*Clause 1’s second half is the part that is easy to get wrong.* The natural implementation discards the stale reading by overwriting the record with it, or by recording it and comparing next time. Both reintroduce the bug one step later: writing  $(t, n)$  back rolls the ledger to size 6, so a later honest observation is compared against a value that should never have been stored. The stale reading must leave no trace.

*Genuine rollback is untouched, and we assert both directions.* An attacker must now present a head that is both newer *and* smaller, which still trips clause 2. The regression test asserts that a smaller-and-older head leaves trust intact and leaves the recorded size at 8, *and* that a smaller head with an equal-or-newer timestamp still reports FORKED. Without the second assertion the guard would be indistinguishable from deleting the check.

### 4.4 Convergence under reordering

Two nodes may receive the same set of signed envelopes in different orders. Any projection that folds them in arrival order will therefore diverge. We found this in a vote handler, which merged by arrival: two nodes receiving the same two signed votes in opposite orders disagreed permanently on both the stored value and the derived score. The fix is to impose a total order using only fields inside the signed envelope — here  $(created\_at\_ms, content\_id)$  — so that every node folds identically regardless of delivery order. The property is asserted over *every permutation* of a conflicting set rather than over one arrival order, because a test that fixes the order cannot distinguish an order-independent fold from a lucky one.

### 4.5 Bridging two islands

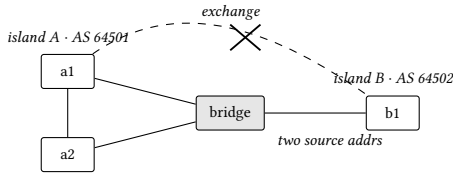
At L3, two ISP islands are severed from each other but a node holds transit on both. That node binds each outbound federation connection to the source address of the corresponding uplink, and relays between islands only when it has a trusted peer on each side. Crossings are quota-accounted per uplink pair rather than per peer, so an island with three nodes is not charged three times for one link’s traffic, and bulk traffic draws on a bucket capped at half the pair’s grant so that a bulk backlog cannot starve emergency broadcasts.

## 5 Adversary and Scope

We assume an adversary that controls routing within its own autonomous system, can withdraw or null-route prefixes, can perform deep packet inspection on transit it carries, can block by SNI, IP or DNS, can throttle selectively, can partition the country or one ISP from another, and can operate its own nodes and federate with honest ones. We assume it cannot break Ed25519 or SHA-256, cannot extract signing keys from an uncompromised device, is not a *global* passive observer, and does not control a majority of the peer set.

Within that model the design resists: content suppression by a server, since content is valid the instant its author signs it and withheld approval is removed as a mechanism rather than policed; tampering by a relay, since signatures cover received bytes and non-canonical encodings are rejected; undetected log rewriting, via gossiped tree heads; write amplification from invalid floods, via the no-write prefix of the pipeline; and loss of reach under partition, via scope-aware path selection and bridging.

We state the exclusions at full strength, because they are numerous and deliberate. There is **no traffic-analysis resistance**: no cover traffic, no mixing, and an observer on both islands can correlate flows. There is **no transport metadata protection**: an ISP sees that a node federates, roughly with whom, and how much. There is **no endpoint-compromise resistance** beyond revocation, and



**Figure 3: The partition testbed.** Islands A and B occupy separate address spaces on networks declared internal, so no route exists between them once the exchange is cut. The bridging node holds transit on both and binds each outbound connection to the corresponding uplink’s source address.

revocation only helps once it reaches peers, which is what the adversary is attacking. Fork detection **assumes an honest observer** exists to gossip a conflicting head. There is **no formal unlinkability proof** between the system’s pseudonymous and identified identity planes; the separation is architectural and unaccompanied by an anonymity-set analysis. Sybil resistance is **deliberately not derived from identity** — cost is the anti-abuse primitive, because requiring identity to stop spam hands the adversary a censorship lever — which bounds the rate of abuse but not the number of identities.

Each exclusion could be bought, and each would be bought with availability or with the ability to run on a single-board computer over an intermittent link. Where a defence and reach conflict, we choose reach and state the limit.

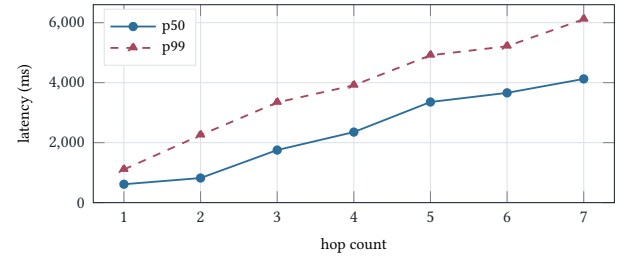
## 6 Evaluation

### 6.1 Testbed

Measurements come from container testbeds on one host. The partition testbed runs four nodes across three networks, two of which are declared internal so that no route exists between them except through the bridging node (Figure 3); each island has its own database and cache, and the islands occupy separate address spaces. The scale testbed generates an  $N$ -node federation chain,  $2 \leq N \leq 16$ , in which node  $i$  is reachable from node 0 by exactly one path of exactly  $i$  hops.

*Why one configuration and not a sweep.* We attempted to sweep chain length and drain interval and discarded the result. One configuration happened to repeat, and it disagreed with itself: an identical four-node, 500 ms run measured 1348 ms at the first hop early in the sweep and 6630 ms later, with delivery falling to 88%. Recreating the host returned it to 888 ms and 100%. The varying quantity was host contention, not the parameter. We therefore report one clean configuration rather than a sweep whose independent variable we could not isolate.

We measure propagation from the timestamp each node records when it stores an envelope, inside the same transaction as the projection. A hop’s latency is therefore a subtraction between two numbers the system wrote for its own reasons, with no observer in the path. This is valid here only because all containers share the host clock, so skew is zero by construction; a geographically distributed deployment would need NTP discipline and an error bar. Our first propagation figure was produced by publishing on one



**Figure 4: Propagation latency by hop count.** Eight-node chain, 200 paced samples, zero loss: all 200 envelopes reached hop 7. Growth is linear in hop count, and the slope is set by the outbox drain interval rather than by the protocol.

node and polling another over HTTP, which reported 623–857 ms; that number is an upper bound charged for a poll interval plus a round trip, and we discarded it.

### 6.2 Cross-implementation agreement

The admission gate of §3.2 is only viable if the accepted encoding is genuinely agreed across implementations; otherwise it becomes a compatibility bug that rejects honest peers. Three independent implementations of the canonical encoder — in TypeScript, Rust and Python, written against the specification rather than ported from one another — agree byte-for-byte on all 16 conformance vectors, including field omission, NFC normalisation, domain separation and plane separation cases. This is what makes strictness affordable, and it is the answer to the natural objection that two implementations will eventually disagree.

### 6.3 Correctness under partition

With the two-node deployment, a post published on one node projects on the other with an identical content identifier and an origin-derived community identifier, across fully independent datas-tores. The federation suite covers 31 assertions spanning admission, quota, streaming, resumable backfill, tree-head gossip with fork detection, and directory exchange.

### 6.4 Multi-hop propagation

Figure 4 reports an eight-node chain, 200 paced samples, with a 1000 ms outbox drain interval. All 200 envelopes reached hop 7, and the seven-hop median is 4125 ms against a p99 of 6115 ms.

Per-hop cost is dominated by the drain interval rather than by the protocol. Predicting  $drain/2 + fixed$  and measuring at two operating points isolates the constant: at a 1000 ms drain the seven-hop median implies 589 ms per hop and a fixed residual of 89 ms; at 250 ms it implies 196 ms per hop and a residual of 71 ms. The residual is stable at **71–89 ms across a fourfold change in the queueing parameter**, and represents the protocol’s real per-hop cost: verify, project, append to the witness log, re-enqueue. Everything above it is a scheduling choice an operator makes for battery and bandwidth reasons.

**We report the offered-load regime with every figure, because it dominates the result.** Publishing the same 200 envelopes as fast as the origin accepted them produced a first-hop median of

10.0 s and a 95th percentile of 173 s, while the *marginal* cost of hops 2–7 remained at 0.4–1.1 s. That shape is queueing and contention on a shared database, not propagation. Both are correct measurements of different questions, and averaging them would describe neither.

## 6.5 Bridged crossing

Crossing an ISP boundary is not one envelope. A newly-published post that a receiving island can render depends on the author’s certificate and the community record, so three envelopes must cross and project in dependency order — a distinction that matters, because measuring only the post conceals where the time goes (§9).

With the exchange severed and the bridge carrying all traffic between islands, the three envelopes arrive on the far island 0.5 s, 1.2 s and 2.1 s after publication. The comparable figures with both islands healthy are 1.5 s, 1.5 s and 1.9 s, so **cutting the exchange does not meaningfully degrade crossing latency** once traffic is routed through the bridge. The container partition gate passes all 19 criteria.

Inside the bridging container, `/proc/net/tcp` shows established federation sockets leaving from *both* configured source addresses, which is the assertion no in-process harness can make. Per-direction accounting confirms the crossing is attributed to the correct uplink pair, and the reserved-capacity check confirms bulk traffic drew on a bucket capped at half the pair grant.

## 6.6 An invalid-envelope flood does not become write load

Section 3 claims that because steps 1–12 perform no database writes, a flood of invalid envelopes costs CPU but cannot amplify into write load. That is a claim about a denial-of-service surface, so we measured it rather than asserting it.

Random bytes would be a weak test: they are refused at parse for a few microseconds and never reach anything expensive. The costly case is a *well-formed* envelope whose signature is wrong, because that one must be parsed, checked for canonicity and then verified before it can be refused. We therefore took a genuine envelope from the node’s own log and mutated one byte inside the signed region, which both invalidates the signature and changes the content identifier — so every request is a novel envelope and none can be short-circuited by deduplication.

Against a single node, 3000 such envelopes at concurrency 16 were absorbed at **730 req/s**, and the node’s database recorded **zero writes**: the envelope log was unchanged, and a collection-level profiler — which captured read operations over the same window, confirming it was live — logged no insert, update or delete. Resident memory fell from 86 MiB to 61 MiB as the garbage collector caught up, and CPU returned to idle immediately.

The rejection histogram is more interesting than the throughput, and we report it because it changes what the number means. Only 9% of the flood reached signature verification; **90% was shed by the per-peer rate limiter first**, before any asymmetric cryptography was performed. The no-write property is therefore real but is the *second* line of defence, and a flood measurement that did not report which step did the rejecting would overstate what the pipeline ordering contributes.

**Table 1: Encoded size, ours versus 80 real ActivityPub activities. Overhead excludes authored text, which is a confound: our fixture carries a 14-character headline while the sampled posts carry 30–524 B.**

|                                       | p50                    | min  | max  |
|---------------------------------------|------------------------|------|------|
| ActivityPub delivered (B)             | 4216                   | 2786 | 6909 |
| of which authored text                | 205                    | 30   | 524  |
| of which JSON-LD context              | 927                    | 927  | 927  |
| <b>ActivityPub overhead (B)</b>       | <b>4015</b>            | 2756 | 6484 |
| ActivityPub, gzipped (B)              | 1074                   | 786  | 1400 |
| Ours, check-in / post / broadcast (B) | 155 / 220 / 243        |      |      |
| <b>Ours, overhead (B)</b>             | <b>155 / 206 / 177</b> |      |      |

*The measurement also found a defect, which is why it was worth doing.* Eight of the 3000 responses were HTTP 500 rather than a typed rejection. The write endpoint decoded untrusted bytes *outside* the pipeline in order to label a metric, and a raw decoder exception is not a typed rejection, so it escaped the error branch — emitting a stack trace per request and letting the caller choose how fast the node filled its own disk. The same bytes arriving over federation were refused politely, so the two ingress paths disagreed about what malformed meant. Decoding for a label now falls back to a constant, and a regression test drives four adversarial encodings that each returned 500 before the change. Fixing it also raised the flood-rejection rate from 440 to 730 req/s, because the node had been spending the difference constructing stack traces — the error path was itself the amplifier.

## 6.7 Read path and footprint

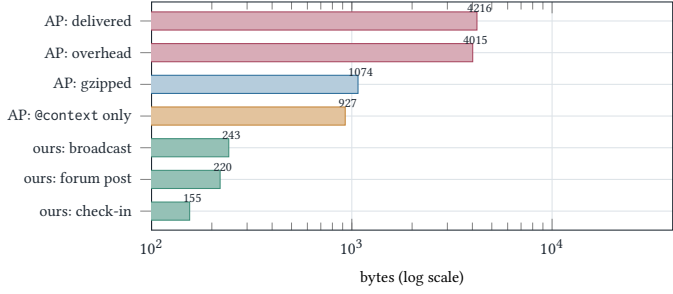
With a keep-alive client in a single process, the read path sustains 228 req/s at concurrency 1 (p50 4.1 ms), 750 req/s at concurrency 8 (p50 9.4 ms) and 688 req/s at concurrency 32 (p50 44.2 ms). We note that measuring this with one process per request instead reported 144 ms, which was timing process creation rather than the server.

An idle federating node holds **62 MiB** resident, rising to **233 MiB** at 54% CPU under a sustained bulk crossing. With the database (147 MiB) and cache (3.7 MiB) co-resident the total is roughly **384 MiB** against a 512 MB single-board-computer target. That is *tight rather than comfortable*, and the database dominates it — a deployment on constrained hardware should expect storage, not the node, to be the binding constraint.

## 6.8 Encoded size against a measured ActivityPub baseline

Encoded envelopes, including a 64-byte signature, are 155 B for a check-in, 220 B for a forum post and 243 B for a Bangla-language broadcast. To make this a comparison rather than an isolated fact we measured **80 real Create/Note activities** from two independent stock ActivityPub instances, taken from actor outboxes so they are as the servers emit them. Because a collection hoists the JSON-LD context to the collection while an activity delivered to a remote inbox carries its own, we re-attached the context each host actually serves on a standalone object.





**Figure 5: Encoded size, ours (teal) against 80 real ActivityPub activities (medians). The JSON-LD @context declaration alone — identical on both sampled hosts — is 4.2× our entire signed forum post. Note the log scale, and that ActivityPub is charged nothing here for authentication while our figures include a 64 B signature.**

Figure 5 and Table 1 report overhead — encoded bytes minus authored text — because content length is a confound and overhead is what a protocol designer controls. On that measure the ratio is 19.5×–25.9×. The JSON-LD context declaration alone, identical on both hosts at 927 B, exceeds our entire signed forum post by a factor of 4.2.

**Two qualifications keep this honest.** First, compression removes most of the advantage: the ActivityPub payload gzips to a 1074 B median, whereas our envelopes do *not* compress — a random 64-byte signature is incompressible and gzip makes all three of ours larger (220 B becomes 243 B). The compressed comparison is therefore 4.4×–6.9×, and we quote both. Second, we charge ActivityPub nothing for authentication, although our figures include a signature that makes the envelope verifiable at rest; deployed ActivityPub authenticates with a transport-level HTTP Signature header that no third party can check against a stored object. A structurally fair comparison would widen the gap, not narrow it. We also measure no HTTP framing on either side, and count one activity where real delivery is per-follower fanout.

This measures per-message cost on a constrained link and nothing else. JSON-LD buys extensibility and semantic interoperation that a fixed schema does not, and we measure none of that.

## 7 Comparison with Related Systems

No quantitative baseline exists for “operates across an ISP partition”, so we compare capabilities. Table 2 scores prior systems on the ladder and on the integrity properties this design turns on. Every cell was checked against the system’s own specification.

Four entries deserve comment because they cut against us. **DTN spans the whole ladder**, and more cheaply, because that is exactly what it was designed for; what it lacks is everything above the bearer. **Both DTN and Serval beat us on the lower rungs**, where our own entry is \* rather than • — we have code but no observation, and they have deployments. **Serval verifies without a server more simply than we do**: its bundle identifier is the verification key, whereas we resolve a certificate. And **Matrix’s redaction preserves more than is commonly assumed** — the

**Table 2: Capability comparison.** • provided and specified, ⊙ partial, optional or non-normative, ○ not provided, \* implemented here but never observed end to end (§8). NF indicates the specification is silent.

|                  | L0–L3 span | L4–L5 span | signed at rest | verify w/o srv | no re-encode | log gossip | tomb-stone |
|------------------|------------|------------|----------------|----------------|--------------|------------|------------|
| DTN / BPv7       | •          | •          | ⊙              | ○              | NF           | NF         | —          |
| Serval (Rhizome) | ⊙          | •          | •              | •              | NF           | NF         | ○          |
| Bridgefy         | ○          | ⊙          | ○              | ○              | —            | —          | —          |
| ActivityPub      | ⊙          | ○          | ⊙              | ○              | ○            | ○          | ⊙          |
| Matrix           | ⊙          | ○          | •              | •              | ○            | ○          | •          |
| AT Protocol      | ○          | ○          | •              | ○              | •            | ○          | ○          |
| <b>This work</b> | •          | *          | •              | •              | •            | •          | •          |

event remains in the graph with its identifier, sender, timestamp, parent links, hashes and signatures intact.

Two entries are frequently misread in the other direction. ActivityPub scores ⊙ on tombstoning because Activity Streams 2.0 [23] does define a Tombstone type and the dominant implementation does keep a table by that name — but nothing serves it, no such object is emitted, and its only reader is a replay guard. The AT Protocol scores ○ on verification without a server because although its commits are signed, neither the signature nor the commit indicates the key type or the public key used; that must be fetched from an external identity directory, which a shutdown removes. It scores ○ on tombstoning by its own explicit statement that record deletion leaves no trace of previous contents [6].

No column in Table 2 is novel on its own. What no prior system holds is all six simultaneously *while spanning L0–L3*, and the engineering cost of that combination — not its novelty — is what this paper reports.

## 8 Limitations

**The lower rungs are unobserved.** L4 and L5 are implemented and unit-tested but have never been exercised end to end. No phone-to-phone mesh has run between two real handsets — the available devices were on different subnets, so local discovery could not see across them — no long-range radio stack has reached a running state on a device, and no physical radio drill has been performed. These are marked \* in Table 2 and no claim in this paper depends on them.

**Everything is measured in containers on one host, and that host is a confound.** Clock skew is zero by construction, the network is a virtual bridge, and one storage engine is shared by several nodes. This is not merely a caveat: as §6 reports, repeating one configuration late in a sweep changed its first-hop median by a factor of five, so this testbed cannot separate a parameter effect from its own contention beyond a single clean run. Chain length and queueing parameter therefore remain unswept, and no wide-area, lossy-link or real-radio measurement exists.

**Unlinkability is architectural, not proven.** The separation between identity planes has no formal argument and no anonymity-set analysis.

**Single-operator testbed.** Every node was configured by us. We have not observed the trust dynamics — probationary admission, vouching, demotion — under independent operators, which is where they matter.

## 9 Lessons

*Bugs have a minimum topology.* The identifier defect of §4 needed two nodes, and the delivery stall below needed three, one of them unreachable while the others stayed up. Neither suite was weak; the topologies were. Relatedly, our in-process partition suite passed all 22 assertions while the container gate failed — the assertions that earned their place are the ones no in-process harness can make, such as reading `/proc/net/tcp` to confirm which source address a socket actually left from.

*An overdue queue entry with zero attempts is a different failure from a failing one.* This is the finding we would most want another team to take away. The container gate failed one criterion — a post crossing after an exchange was cut — and the standing explanation was that the crossing was merely slow. Measuring the three-envelope dependency chain instead of the post showed the crossing cost 3.2 s before the cut and 407.6 s after it, and that 406.5 s of that was the origin node failing to hand off to the bridge, while the bridge itself relayed in 1.1 s throughout. During the stall every outbound row sat at zero attempts while overdue by 150 seconds. Retries that fail tell you about the destination; work never attempted tells you about the scheduler.

The cause was that one drain pass delivered to peers serially with no deadline. A severed link blackholes rather than refuses, so a connection attempt to the unreachable peer neither succeeded nor errored, and every peer behind it in the same pass waited — including the bridge, whose entire purpose is to route around the partition that caused the stall. A node with three peers degraded to the availability of its worst peer instead of its best, inverting what multi-homing is supposed to buy.

Delivering to peers concurrently and bounding each attempt with a deadline reduced the post-cut crossing from **407.6 s to 2.1 s**, a factor of 195, and took the container partition gate from 18 of 19 criteria to all 19. A two-peer deployment could never have exposed this: with one peer there is no head of line to block, which is why it survived both the single-node and the two-node phases and was caught only when a third peer became unreachable while the others stayed up.

## 10 Conclusion

Availability under partition is a design ordering, not a feature. Holding it first forces specific and sometimes uncomfortable choices: identifiers derived from signed bytes rather than from the node that stores them, admission cost charged once at the origin rather than re-verified on receipt, canonical encoding used to admit rather than to verify, and a rejection where a more forgiving system would repair. We have shown that this composition operates from global reach down to bridged ISP islands, measured what it costs in latency, memory and bytes, and reported a partition-availability defect that only a deployed multi-peer testbed could expose. The rungs below the ISP island remain implemented and unobserved, and closing that gap on real devices and real radios is the work we think matters next.

## References

- [1] Martin R. Albrecht, Jorge Blasco, Rikke Bjerg Jensen, and Lenka Mareková. 2021. Collective Information Security in Large-Scale Urban Protests: the Case of Hong Kong. In *30th USENIX Security Symposium*.
- [2] Martin R. Albrecht, Jorge Blasco, Rikke Bjerg Jensen, and Lenka Mareková. 2021. Mesh Messaging in Large-Scale Protests: Breaking Bridgefy. In *Topics in Cryptology – CT-RSA 2021*. 375–398. doi:10.1007/978-3-030-75539-3\_16
- [3] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. 2012. High-Speed High-Security Signatures. *Journal of Cryptographic Engineering* 2, 2 (2012), 77–89. doi:10.1007/s13389-012-0027-1
- [4] Edward J. Birrane and Kenneth McKeever. 2022. *Bundle Protocol Security (BPsec)*. Technical Report RFC 9172. Internet Engineering Task Force. doi:10.17487/RFC9172
- [5] Edward J. Birrane, Alex White, and Sarah Heiner. 2022. *Default Security Contexts for Bundle Protocol Security (BPsec)*. Technical Report RFC 9173. Internet Engineering Task Force. doi:10.17487/RFC9173
- [6] Bluesky Social. 2026. AT Protocol Repository Specification. <https://atproto.com/specs/repository>.
- [7] Scott Burleigh, Kevin Fall, and Edward J. Birrane. 2022. *Bundle Protocol Version 7*. Technical Report RFC 9171. Internet Engineering Task Force. doi:10.17487/RFC9171
- [8] Laurent Chuat, Pawel Szalachowski, Adrian Perrig, Ben Laurie, and Eran Messeri. 2015. Efficient Gossip Protocols for Verifying the Consistency of Certificate Logs. In *2015 IEEE Conference on Communications and Network Security (CNS)*. 415–423. doi:10.1109/CNS.2015.7346853
- [9] Cosmos SDK. 2020. ADR-020: Protocol Buffer Transaction Encoding. <https://github.com/cosmos/cosmos-sdk/blob/main/docs/architecture/adr-020-protobuf-transaction-encoding.md>.
- [10] Rasmus Dahlberg and Tobias Pulls. 2018. Aggregation-Based Certificate Transparency Gossip. In *Proceedings of the 13th International Conference on Availability, Reliability and Security (ARES)*. doi:10.1145/3230833.3230869
- [11] Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda. 2018. Privacy Pass: Bypassing Internet Challenges Anonymously. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2018. 164–180. doi:10.1515/popets-2018-0026
- [12] Kevin Fall. 2003. A Delay-Tolerant Network Architecture for Challenged Internets. In *Proceedings of the 2003 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM)*. 27–34. doi:10.1145/863955.863960
- [13] fiatjaf. 2023. NIP-01: Basic Protocol Flow Description. <https://github.com/nostr-protocol/nips/blob/master/01.md>.
- [14] Alan Ford, Costin Raiciu, Mark Handley, Olivier Bonaventure, and Christoph Paasch. 2020. *TCP Extensions for Multipath Operation with Multiple Addresses*. Technical Report RFC 8684. Internet Engineering Task Force. doi:10.17487/RFC8684
- [15] Paul Gardner-Stephen. 2011. The Serval Project: Practical Wireless Ad-Hoc Mobile Telecommunications. In *Rural, Remote and Humanitarian Telecommunications*.
- [16] Google. 2024. Protobuf Serialization Is Not Canonical. <https://protobuf.dev/programming-guides/serialization-not-canonical/>.
- [17] Ari Keranen, Christer Holmberg, and Jonathan Rosenberg. 2018. *Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal*. Technical Report RFC 8445. Internet Engineering Task Force. doi:10.17487/RFC8445
- [18] Ben Laurie, Adam Langley, and Emilia Kasper. 2013. *Certificate Transparency*. Technical Report RFC 6962. Internet Engineering Task Force. doi:10.17487/RFC6962
- [19] Moxie Marlinspike and Trevor Perrin. 2016. The X3DH Key Agreement Protocol. <https://signal.org/docs/specifications/x3dh/>.
- [20] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology – CRYPTO ’87*. 369–378. doi:10.1007/3-540-48184-2\_32
- [21] MITRE. 2024. CVE-2024-23832: Insufficient Origin Validation in Mastodon. In *Common Vulnerabilities and Exposures*. <https://nvd.nist.gov/vuln/detail/CVE-2024-23832>.
- [22] Open Observatory of Network Interference. 2024. Bangladesh: Internet Shutdowns and Social Media Blocking Amid Student Protests. <https://ooni.org/post/2024-bangladesh-internet-shutdowns/>.
- [23] James M. Snell and Evan Prodromou. 2017. Activity Streams 2.0. W3C Recommendation. <https://www.w3.org/TR/activitystreams-core/>.
- [24] Dominic Tarr, Erick Lavoie, Aljoscha Meyer, and Christian Tschudin. 2019. Secure Scuttlebutt: An Identity-Centric Protocol for Subjective and Decentralized Applications. In *Proceedings of the 6th ACM Conference on Information-Centric Networking (ICN)*. 1–11. doi:10.1145/3357150.3357396
- [25] The Matrix.org Foundation. 2024. Matrix Specification, Version 1.11. <https://spec.matrix.org/v1.11/>.
- [26] Christopher Lemmer Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou. 2018. ActivityPub. W3C Recommendation. <https://www.w3.org/TR/activitypub/>.